Socket Communication and Vision Data Parsing

Meituan Challenge Robot Code Guide 03 | English version | 2026-10-13

1 Overview

The `Connect_socket` module in `connect_socket.docx` handles communication between the robot, vision system, and PLC. The camera branch sends string commands, receives vision results, and parses them into shared numeric values. The PLC branch provides register read/write and handshake control procedures. The main battery pick-and-place chain explicitly selects the camera branch; the presence of PLC procedures in the file does not mean that PLC operations run during every battery task iteration.

`socket1/socket2`, the send/receive strings, numeric fields, and most state variables are defined in `Modul1`. The declarations with matching names at the beginning of the communication file are mostly commented out.

2 Connection Dispatch and Camera Requests

`Connect2` selects mutually exclusive branches based on two Boolean values:

socket_start	startPLC	Behavior
TRUE	FALSE	Establish a socket1 connection and call SocketConnectCamara
FALSE	TRUE	Establish a socket2 connection and call ConnectPLC
Other combinations	Other combinations	The current ELSE branch performs no operation

The original program contains device addresses and ports for the workstation network. They must be checked against the actual network configuration during deployment. This guide does not reproduce internal network addresses or treat them as general-purpose configurations.

The normal camera-processing chain is:

```
Battery_get_values
  -> send_string := input_string
  -> socket_start := TRUE; startPLC := FALSE
  -> Connect2
       -> establish camera connection
       -> SocketConnectCamara
            -> SocketSend sends send_string
            -> SocketReceive receives into receive_string_copy
            -> save receive_string and string length
            -> close both sockets
            -> ProcessReceivedData
                 -> reset nval1..nval6
                 -> parse fields and print debug values
                 -> CamaraFinish := TRUE
```

`Getvalues` requests multiple-target coordinates, while `Redvalues` requests a single red target. The current code creates a connection for each request and closes it after receiving data. It does not show camera-session management that can be described as a persistent connection.

3 Data Fields and Parsing

Fields	Meaning in the three-color flow	Use in the single-red-target flow
nval1, nval2	Blue X, Y	Red X, Y, used for picking and placing
nval3, nval4	Green X, Y	The third field is labeled as an angle in a comment; the pick-and-place logic does not use it to adjust orientation
nval5, nval6	Yellow X, Y	No corresponding use in single-color picking and placing

`ProcessReceivedData` first sets all six numeric values to zero, then extracts fields from `receive_string_copy`. The code attempts to skip a colon-delimited prefix before the first comma, distinguishes fields using commas and semicolons, and handles double semicolons between groups. Each value is written to a shared variable through `StrToVal`.

The following intended formats can be inferred from the branch structure. They are illustrations for interface checks, not protocol samples validated on a controller:

```
Single target: X,Y,R
Three targets: Xb,Yb;;Xg,Yg;;Xy,Yy
```

Whether the actual output includes a prefix, trailing separators, empty fields, or an angle must be checked item by item against the VisionMaster formatting node and real replies. No complete vision-project configuration or packet-capture data is available, so this guide does not claim that the parser supports all formats or malformed data.

`ok` receives the return value of `StrToVal`, but the code does not use it to reject invalid results field by field. `CamaraFinish = TRUE` means the procedure has reached its end; it does not confirm that every field was converted successfully. The behavior of `StrFind` when a separator is absent, and the subsequent substring boundaries, also require verification on the target RobotWare version.

4 PLC Register Branch

`ConnectPLC` sets the read/write quantity to 1 and selects an operation based on the shared variable `address`:

Condition or register	Current code behavior
address = 3000	Set WriteStart to 3000 and write the value 1
address from 2101 through 2104	Write address - 2100 to 2100
Register 2020	Write the value 1 as the synchronization flag described in the code comments
Register 2000	Read repeatedly until Readnum{1} is nonzero

The equipment meanings above come from code comments: 3000 corresponds to the conveyor cylinder, 2100 to a rotary-table angle code, and 2000 to a status permitting robot motion. The actual PLC program and register map are not supplied, so the on-site configuration must be verified further. After the loop exits, the code only sets `WriteStart = 2000`; the adjacent reset and write statements are commented out. It therefore cannot be described as having cleared that register.

On normal completion, the procedure sets `PlcFinish = TRUE`, resets `address` and `startPLC`, and closes both sockets. If the value read from 2000 remains zero, the current wait loop has no explicit iteration or time limit.

5 Message Helper Procedures

- `ReadHoldingRegister` assembles a 12-byte read request with function code `0x03`, sends it, receives a byte array, and then calls `ByteForWord`.

- `ByteForWord` combines two bytes from the response. The current loop decodes only one register value and assigns it to `Readnum`.

- `WriteHoldingRegister` calls `WordForbyte` to pack the write request, then sends it and receives a reply.

- `WordForbyte` uses the write-multiple-registers function code `0x10` and splits the address, quantity, and data into high and low bytes. The current transmit array has a fixed length of 15 bytes, matching the existing single-register use; it cannot be extended directly to write an arbitrary number of registers.

This byte layout reflects Modbus TCP-style register communication, but the material does not include a PLC interface specification, response validation, or integration-test records. The read procedure uses the global `ReadStart`, and the unit identifier in the message is fixed. The parameters in the procedure signatures should therefore not all be described as fully determining the request content.

6 Error Handling and Integration Checks

The `ERROR` section in `Connect2` resets the communication-selection flags and `address`, then closes the sockets. No explicit reconnection count, structured error reporting to the caller, or unified clearing of all vision results was found. `SocketConnectStatus` and `all_socket` also exist in the module, but they do not appear in the supplied battery call chain.

Integration checks should focus on camera request and reply formats, separator-search boundaries, handling of numeric conversion failures, isolation of stale values after network errors, PLC response fields, and conditions for exiting waits. Before execution, the initialization, tool, workobject-coordinate, and I/O configurations listed in the main-program guide must be supplied.

This guide is a static code interpretation. No network interfaces were exercised, no robot or PLC was connected, and no controller compilation, simulation, or physical robot testing was performed.

7 Source References

Source version: `lkkkkk667/meituan`, commit `68e262bbedcf2269ba3a7321a045f97e96f185b6`.

In `connect_socket.docx`: `Connect2`, `SocketConnectCamara`, `ProcessReceivedData`, `ConnectPLC`, `ReadHoldingRegister`, `ByteForWord`, `WriteHoldingRegister`, and `WordForbyte`. See `Modul.docx` for shared variables and `battery.docx` for camera-command settings.

Related guides: Main Program · Battery Task.